Unit-06
" Collection Framework in Java"

Topic List :-

- ❑ Introduction to collection framework.
- ❑ List Interface , Array List , Linked List.
- ❑ Set interface , hash set , Tree set .
- ❑ Map Interface , HashMap, Tree Map.
- ❑ Iterators and Generics.

Topic –01
"Introduction to Collection Framework"

- ✓ Collection Framework is the collection of different data structure and interfaces/classes used to work with them.
- ✓ Collection Framework provides ready-made data structures for storing and Manipulating groups of objects.
- ✓ Collection Framework improve productivity by making code more reusable , maintainable and faster to develop.
- ✓ In Java Collection framework provide ready-made data structure which can been directly used as per our requirement we do not need to make data structure from scratch.
- ✓ Collection Framework useful when there is Time-Boundation .
- ✓ In java we cannot make an object of an interfaces so, we can these interfaces as an classes.

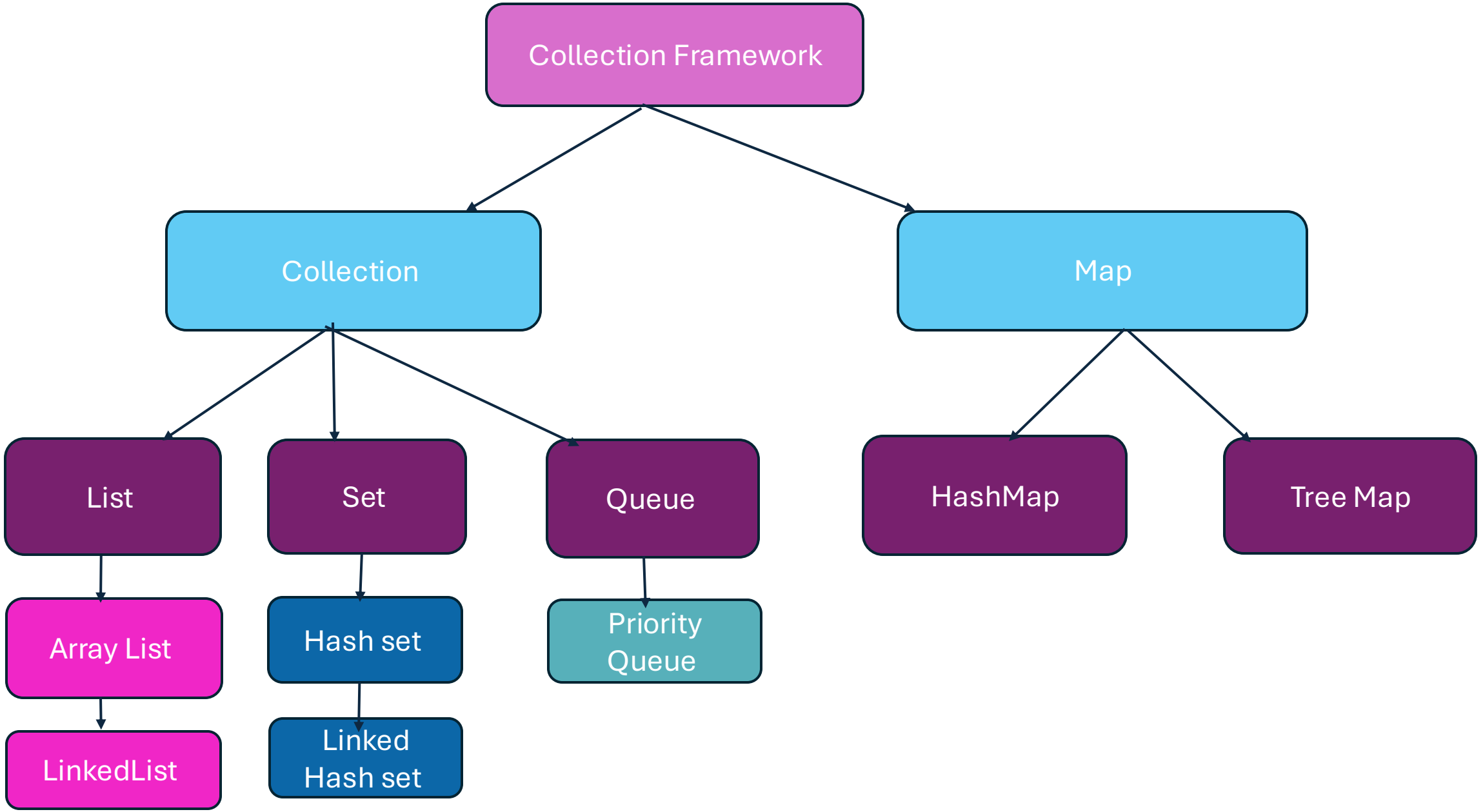
Why We need Collection framework ?

We need Collection framework because plain arrays are rigid and primitive, whereas the collection framework provides a standardized , ready-to-use architecture of dynamic data structure and high-performance algorithm. In short it is combo of classes and interfaces.

Main Methods in collection Framework :

- Add – add method is used to add any type of information in object which is created from particular class.
- Size – size method is used to find the size of any object, collection or data structure.
- Remove – remove method is used to remove an object.
- Iterate – Iterate means applying loops on objects .
- Add all – Add all method is used to combine multiple objects into particular data structure.
- Remove all – reverse process of add all in this we ca remove multiple objects from particular data structure.
- Clear – clear method is used to completely clear / remove our data strucutre .

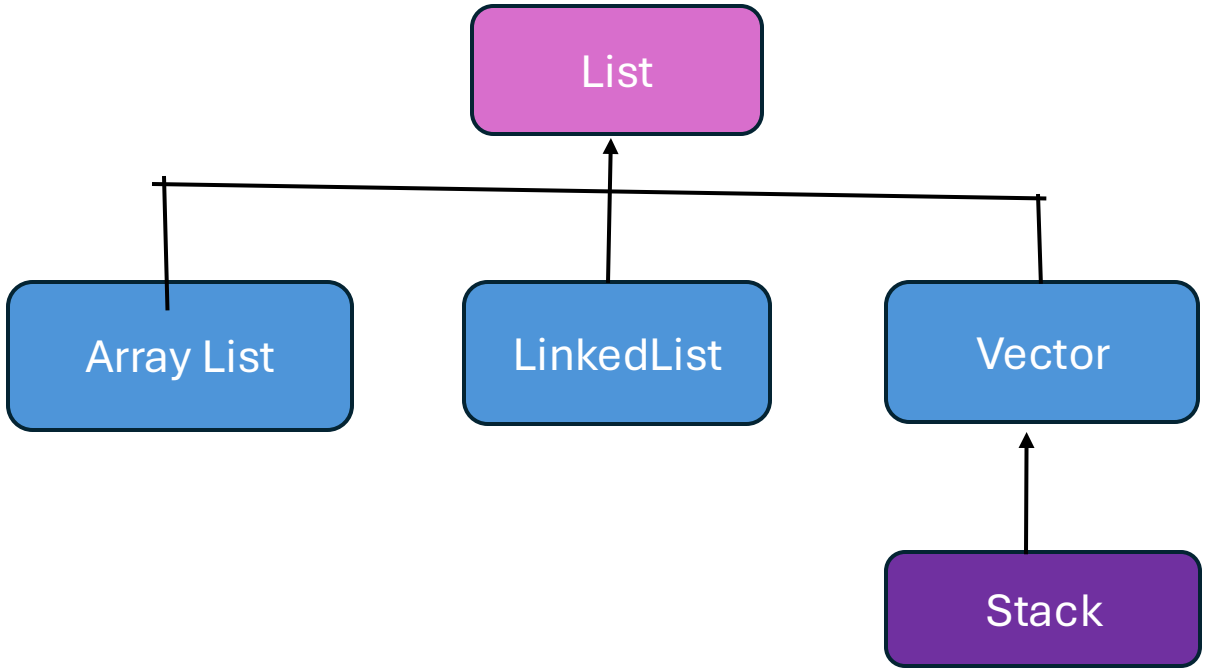
Main Component of collection framework :-



Topic :-02
"List Interface, Array list and Linked list"

List Interface :-

- Easiest to implement List Interface in java.
- It consist of three main classes .



Array List

- Array list is of variable size means it can be extends (increase) or intended (Decrease) .
- In Array list Elements are stored in non-contiguous memory form .
- As Compare to Array in array we can store primitive datatypes (int, float, char) and object as well but in array list we can only store objects.
- Array list is created in heap memory form.

Syntax of Creating ArrayList in java collection Framework:

ArrayList<wrapperclass> reference value = new ArrayList<wrapperclass>();

Operations/methods can be performed on Array list :

- Add : Adding an Element in the Array list.
- Get : Getting already added element.
- Modify: Add in between or alter the elements.
- Delete : Also known as remove used to delete or remove already added element.
- Iterate : Applying iterations (loops) on Elements or list.

Practical Implementation of Array list in java:-

```
import java.util.ArrayList;
import java.util.Collections;
public class Main
{
   public static void main(String[] args)
   {
      ArrayList<Integer>list=new ArrayList<Integer>();
      //adding element
      list.add(0);
      list.add(1);
      list.add(2);
      System.out.println(list);

     //get element
      int element = list.get(0);
      System.out.println(element);
```

```
//add Element in between
list.add(1, 1);
System.out.println(list);

//set element that is already defined in java
list.set(0 ,5);
System.out.println(list);

//delete element
list.remove(3);
System.out.println(list);

//size of list
int size = list.size();
System.out.println(size);
```

```
    //Applying loops in list
    for(int i=0; i<list.size(); i++)
    {
       System.out.println(list.get(i));
    }
    System.out.println();

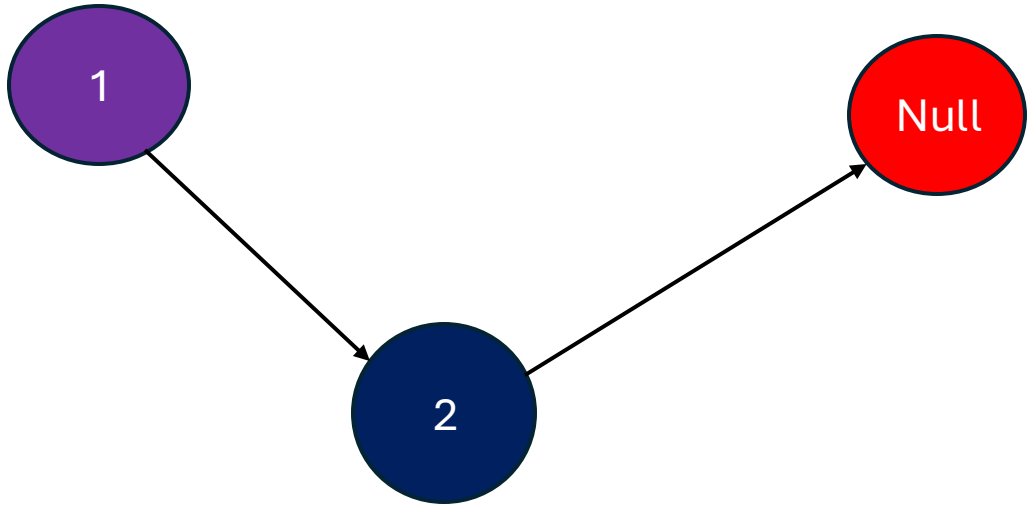
    //sorting elements
    Collections.sort(list);
    System.out.println(list);

  }
}
```

LinkedList(LL)

- ✓ LinkedList is Also Variable size means it do not have any fixed size .
- ✓ Combining different – different parts of memory (Heap).
- ✓ Linking multiple data with each other through notes .

Visualization :



Syntax for creating LinkedList in java collection framework:

LinkedList<wrapperclass(datatypes)>reference value = new LinkedList<wrapperclass>();

Time complexity conversion between Arraylist and Linkedlist

Parameters	ArrayList		LinkedList
Insert	O(n)	>	O(1)
Search	O(1)	<	O(n)

For Optimizing Time-Complexity in both cases we use Linked List for perfoming manipulation tasks such as addition of node on the other hand we use Array list when performing searching operation .

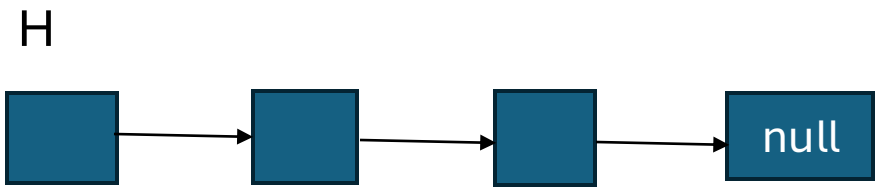
Properties of Linked List :

- ❑ Variable Size – means size is not fixed adjust size as per our requirement .
- ❑ Non-contiguous – means we want chunks that can be linked with each other.
- ❑ Insert in O(1).
- ❑ Search in O(n).

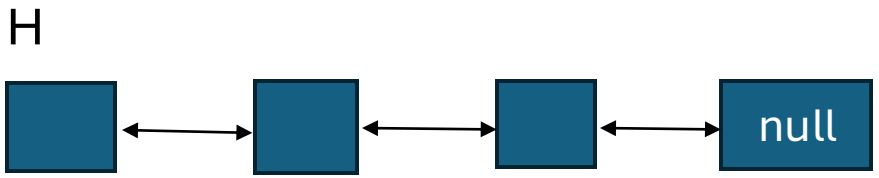
Real world Example of Linked List is Train .

Types of Linked List :

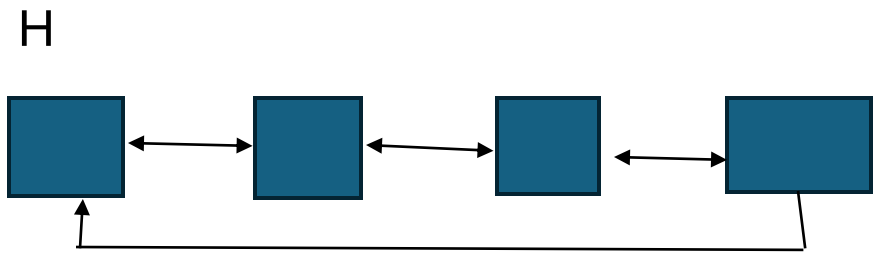
Singular Linked List -



Double Ended Linked List -



Circular Linked List -



Practical implementation of Linked List in java :-

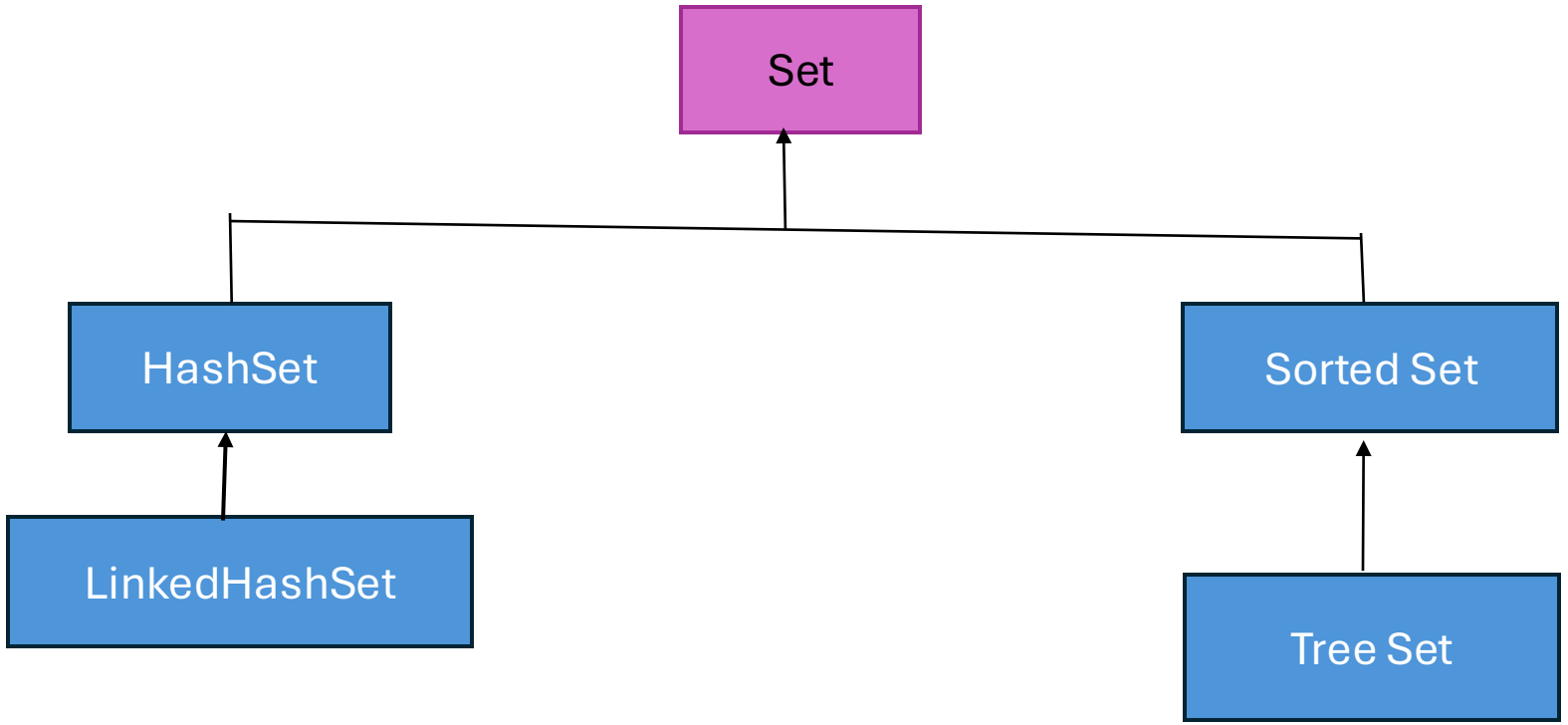
```
import java.util.*;
class LL
{
   public static void main(String[] args)
   {
      LinkedList<String> list = new LinkedList();
      list.addFirst("a");
      list.addFirst("is");
      System.out.println("list");
      list.addFirst("this");
      list.addLast("list");
      System.out.println("list");
      System.out.println(list.size());
      for(int i=0; i<list.size(); i++)
      {
         System.out.print(list.get(i) +"->");
      }
```

```
System.out.println("null");
        list.removeFirst();
        System.out.println(list);
        list.removeLast();
        System.out.println(list);
    }
}
```

Topic :-03
" Set Interface , Hash set , Tree set

Set Interface :-

- ❑Set Interface is one of the most important interface in java .
- ❑Set is an group of objects and all objects is unique.
- ❑In set interface it removes duplicates value and store only unique value inside it .



HashSet

- It is Important data structure in collection framework.
- It is mostly used when we want to write efficient code and reduce time complexity.
- Hash set is popular data structure due to its time complexity.

Insert/add -O(1).

Search/contains-O(1).

Delete/remove-O(1).

- Iterator is used to iterate the set elements.
- Set does not have any kind of index number so with the help of iterator we can iterate the set.
- Iterator have two main functions -

    # Next – print element one by one.

    # has next – It return true and false.

Practical implementation of HashSet in java collection framework:-

```
import java.util.HashSet;
import java.util.Iterator;
public class Hashing
{
   public static void main(String[] args)
   {
      HashSet<Integer>set = new HashSet<>();
      //insert element in set
      set.add(1);
      set.add(2);
      set.add(3);
      set.add(1);
```

```
// Remove element
      set.remove(2);
      System.out.println(set);


      // Access elements one by one
      Iterator<Integer> Iter = set.iterator();


      while(Iter.hasNext())
      {
         System.out.println(Iter.next());
      }
   }
}
```

```
//search - contains
      if(set.contains(1))
      {
         System.out.println("set contains 1");
      }
      if(!set.contains(6))
      {
         System.out.println("Does not contains 6");
      }
//Delete
      set.remove(1);
      if(!set.contains(1))
      {
         System.out.println("does not contains 1 - we deleted 1");
      }
```

```
//size of set
      System.out.println("Size of set is : " +set.size());

      //print all elements of set
      System.out.println(set);

      //Iterator in  set
      Iterator it = set.iterator();
      while(it.hasNext())
      {
         System.out.println(it.next());
      }
   }
}
```

Linked Hashset

- Linked Hashset is a child class of Hashset.
- It is very similar to hashset except the following differences.
  - The Underlying Data Structure is a combination of LinkedList and Hashtable.
  - Insertion order is preserved , means in which format we are inserted elements in linked hashset in the same order we will get if we print elements. This is Exception in Linked Hashset.
- It will not allow Duplicates values .

Practical Implementation of Linked Hashset in java collection framework :

```
import java.util.LinkedHashSet;
import java.util.Iterator;


public class Main
{
   public static void main(String[] args)
   {
      LinkedHashSet<String> set = new LinkedHashSet<>();
```

```
// Inserting elements
        set.add("java");
        set.add("is");
        set.add("Simple");
        set.add("to");
        set.add("understand");

        // Insertion order is preserved
        System.out.println(set);

        // Duplicate value
        set.add("java");
        System.out.println(set);

        // Size of LinkedHashSet
        System.out.println(set.size());
```

Tree Set

- Tree Set is a collection that stores elements in sorted manner we don't need to apply sorting function on it as compare to array list .
- Does not allow null elements because sorting is based on comparison , which may cause null pointer Exception .
- It is not Synchronized and can be made synchronized using **Collections.synchronizedSet().**

Syntax :

TreeSet<wrapper class> Reference name = new TreeSet<>();

Important Tree Set methods :

Lower() - Returns the greatest elements strictly smaller than the given elements.

System.out.println(set.lower(30)); o/p 20

Higher() - Returns the smallest elements strictly greater than the given element .

System.out.pritnln(set.higher(30)); o/p 40

Floor() - Returns the greater element less than or equal to the given value.

System.out.println(set.floor(30)); o/p 30

Ceiling() - Returns the smallest element greater than or equal to the given value.

System.out.println(set.ceiling(35)); o/p 40

Practical Implementation of Tree Set in java collection framework :

```
import java.util.TreeSet;


public class Main{
   public static void main(String[] args)
   {
      TreeSet<Integer>TS=new TreeSet<Integer>();
      TS.add(50);
      TS.add(20);
      TS.add(80);
      TS.add(10);
      TS.add(30);
      TS.add(70);
      TS.add(90);

      System.out.println(TS);
```

```
//duplicate value
    TS.add(30);
    System.out.println("Tree set : " +TS);

    //size
    System.out.println("size : " +TS.size());

    //contains
    System.out.println("containing 30 : "+TS.contains(30));

    //first()
    System.out.println("First element : " +TS.first());

    //Last()
    System.out.println("Last Element : " +TS.last());
```

```
    //lower()
     System.out.println("lower than 50 : " +TS.lower(50));

    //higher()
    System.out.println("higher than 50 : " +TS.higher(50));

    //floor()
    System.out.println("floor of 55 : " +TS.floor(55));

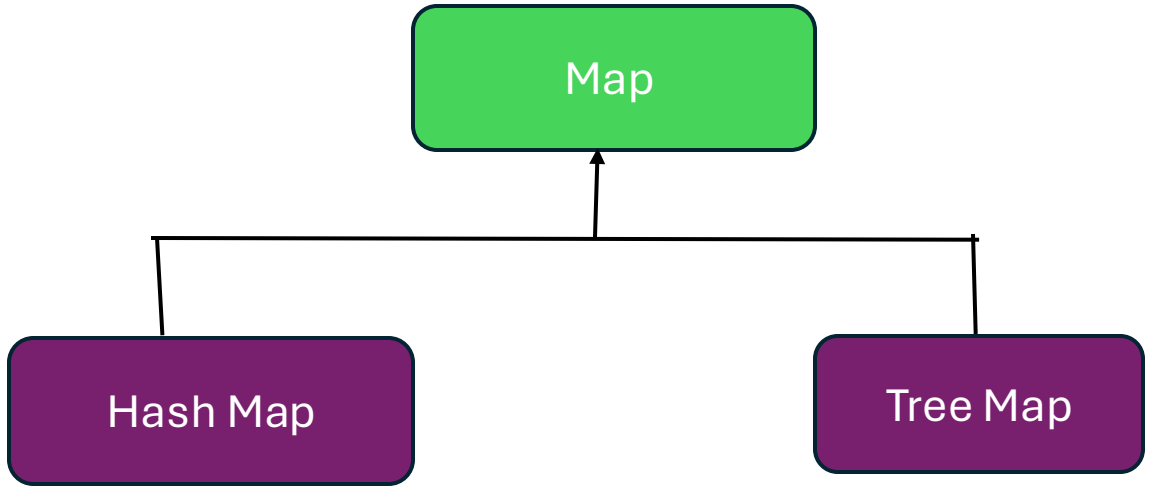
    //ceiling
    System.out.println("Ceiling of 55 : " +TS.ceiling(55));

    //remove()
    TS.remove(30);
    System.out.println("After removing 30 : " +TS);
  }
}
```

Topic :-04
"Map interface , HashMap , Tree map "

Map Interface :-

- Map interface stores elements into **"key - value pair"** and it is used to map two things "Key" and "value".
- Map interface is implemented using set interface .
- In map interface as it stores elements in key-value pair so key must be unique and value can be duplicates .
- Hash map allows one key null , but tree map does not allow null keys .



Main Methods used in Map interface :

- **Put() :** Add data into map .

  Ex:- map.put(101, Dhruvil);
- **Get() :** Retrieve value from the existing map.

  EX:-System.out.println(map.get(101));
- **Remove() :** Delete entry from the map .

  Ex:-map.remove(102);
- **Contains key():** checks whether particular key is exist or not

  EX:-System.out.println(map.containskey(101));
- **Contains value():** checks whether particular value is exist or not .

  EX:-System.out.println(map.containsValue("Dhruvil"));
- **Size():** Returns the size of key-value pair

  EX:-System.out.println((map.size());
- **IsEmpty():** Checks whether map contains anything.

  EX:- System.out.println((map.isEmpty());

HashMap

- HashMap stores elements in key-value pair . It is an part of java collection framework.
- In HashMap key are unique but value can be duplicates.
- Internally it uses hashing , hence allow efficient key-based retrieval, insertion and deletion with average O(1)time.
- HashMap stores one key and multiple null values . If null key is added multiple time then it will overwrite the previous value.

Syntax :

```
HashMap<wrapperclass> reference value = new HashMap<>();
```

entryset() : Returns all the "key-value pair "

System.out.pritnln(map.entryset());

Practical implementation of HashMap in java collection:

```
import java.util.HashMap;
import java.util.Map;
public class Main
{
   public static void main(String[] args)
   {
      HashMap<Integer, String> students = new HashMap<>();


      // Add students
      students.put(101, "Amit");
      students.put(102, "Rahul");
      students.put(103, "Neha");
      students.put(104, "Priya");

     // Display HashMap
      System.out.println("Students: " + students);
```

```
// Get student
    System.out.println("Student with ID 102: " + students.get(102));

    // Check key
    if(students.containsKey(103))
    {
       System.out.println("Student ID 103 exists");
    }

    // Update student
    students.put(102, "Rahul Kumar");

    // Remove student
    students.remove(104);

    //size of HashMap
    System.out.println(students.size());
```

```
    // Display final data
    System.out.println("Final Students:");

    for(Map.Entry<Integer, String> entry : students.entrySet())
        {
          System.out.println( "ID: " + entry.getKey() +", Name: " + entry.getValue());
        }
  }
}
```

Tree Map

- ✓ Tree map stores elements same as hash map it store elements in the form of "key – value pair ".
- ✓ Tree map stores key-value pair in sorted manner . If we insert key value pair in unsorted manner but when we print tree map it will return key value manner in sorted order.
- ✓ Tree map does not allow null key but it allows null value .

Practical implementation of Tree Set in java collection Framework :

```
import java.util.TreeMap;
import java.util.Map;
public class Main
{
   public static void main(String[] args)
   {
      TreeMap<Integer, String> students = new TreeMap<>();
```

```
// Add students
students.put(103, "Amit");
students.put(101, "Rahul");
students.put(104, "Neha");
students.put(102, "Priya");
 // Display TreeMap
System.out.println("Students: " + students);


// Get student
System.out.println("Student with ID 102: " + students.get(102));

//Returns the lastkey
System.out.println(students.lastKey());

//returns First entry of treemap
System.out.println(students.firstEntry());
```

```
//returns the last entry of TreeMap
    System.out.println(students.lastEntry());


    // Check key
    if(students.containsKey(103))
    {
       System.out.println("Student ID 103 exists");
    }


    // Update student
    students.put(102, "Rahul Kumar");


    // Remove student
    students.remove(104);


    //size of TreeMap
    System.out.println(students.size());
```

```
// Display final data
      System.out.println("Final Students:");


      for(Map.Entry<Integer, String> entry : students.entrySet())
      {
         System.out.println("ID: " + entry.getKey() +", Name: " + entry.getValue());
      }
   }
}
```

Topic :-05
"Iterator and generics"

Iterator :-

An Iterator in java is one of the most common used cursor in java collection framework . It is used to iterate through elements of collection one by one.

It is used to traverse in forward direction only .

Removes elements safely using function Remove().

Declaration of Iterator : -

Public interface Iterator<E>

Methods of iterator interface :-

Has Next : Returns True or False if the iteration has more elements.

Next : Returns the next elements in the iteration .